

개발자, 홍지상



 010-3347-2408

 nobledjs@naver.com

 <https://github.com/jisangjiha>

목차

01

역량

02

팀 프로젝트

짬랑 Zzalang
만두 Mando

03

개인 프로젝트

Omok
do it;
베이글 프로젝트

Frontend

- HTML & CSS: 시맨틱 마크업과 CSS Flexbox 및 Grid 레이아웃을 활용하여 반응형 웹 페이지를 구현한 경험
- CSS module & SCSS: CSS Module과 SCSS를 결합해 컴포넌트 기반의 스타일링을 구축하고, 유지보수성을 향상시킨 경험
- JavaScript: 비동기 프로그래밍을 활용하여 API 통신을 처리하고, 모듈화를 통해 코드의 재사용성을 높인 경험
- TypeScript: 타입 선언을 통해 변수와 함수의 타입을 명시하여 코드의 안정성을 향상시킨 경험
- React: React Hook과 상태 관리를 활용하여 SPA를 개발할 수 있으며, Context API를 통해 전역 상태를 효율적으로 관리한 경험
- Next.JS: SSR을 적용하여 초기 로딩 속도를 개선하고 파일 기반 라우팅 시스템을 활용하여 직관적인 페이지를 관리한 경험
- Chakra UI: 기본적인 UI 컴포넌트를 활용해본 경험
- Axios: RESTful API와의 통신을 수행하고, 비동기 데이터 처리를 경험
- Vite: 개발 서버를 구축하고, 빠른 모듈 번들링을 경험
- Storybook: 컴포넌트 단위의 UI 개발 및 테스트를 위해 활용한 경험
- Playwright: E2E 테스트 자동화 스크립트를 작성하여 사용자 시나리오 기반의 테스트를 수행한 경험

Collab Tool

- VS code: 다양한 확장 프로그램과 단축키를 활용하여 개발 효율성을 향상시킨 경험
- Git & Github: 버전 관리 및 코드 관리를 수행한 경험
- Notion, Figma, Slack: 팀원과의 커뮤니케이션을 수행하기 위해 간단한 기능 구현 경험

1. Zzalang

게시판 서비스

개발 기간
(진행 중)

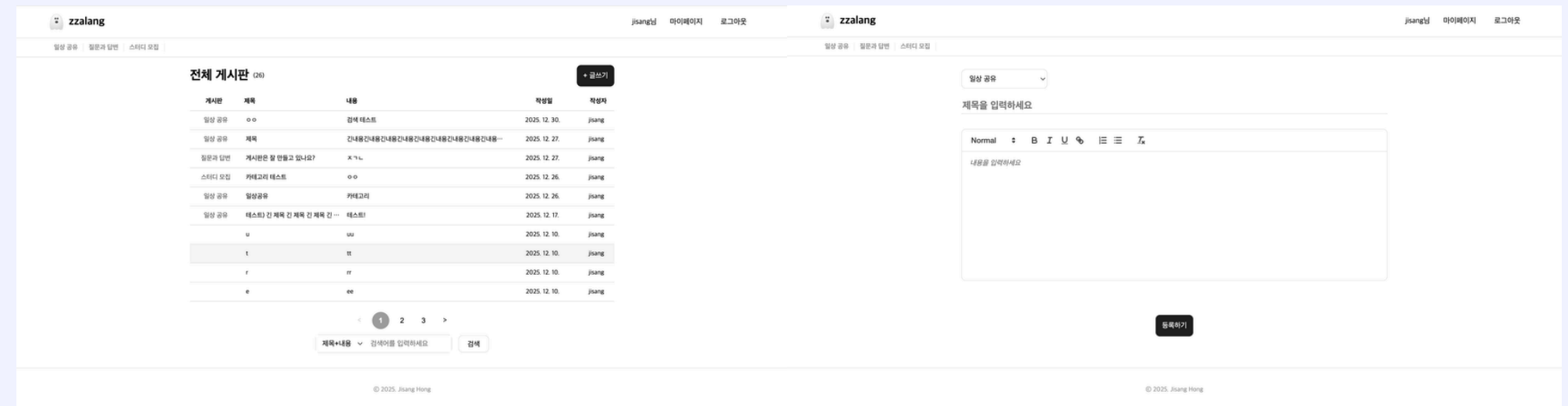
팀원 구성
FE 1, BE 1

사용 스택

- HTML, CSS module, TypeScript, React
- VScode, Git, Github

Github

<https://github.com/jisangjiha/zzalang>



서비스 목적

: 백엔드 API와 연동된 게시판 서비스를 구현하며, 데이터 기반 UI 설계와 협업 경험을 쌓기 위한 팀 프로젝트

기능 분류

- 인증 상태에 따른 화면 제어
- 목록 데이터 페이지네이션 처리
- 검색 조건에 따른 서버 데이터 요청 구조 설계

2. Mando

작업 기여도

1. 인증 기반 화면 흐름 설계 및 API 연동

: 사용자의 인증 상태에 따라 게시물 작성 버튼 노출을 제어하여 접근 가능한 페이지가 달라지도록 구현함.

2. 게시물 목록 페이지네이션 구조 설계

: 게시물 목록 조회 시 서버에서 전달받는 페이지 정보를 기준으로 페이지네이션 UI를 구성함.

3. 검색 기능 UI-API 연계 구현

: 게시물 검색 기능을 구현하고 페이지네이션도 함께 동작하도록 상태 구조를 정리함.

트러블 슈팅

1. 카테고리 필터링 구현 및 상태 관리 개선

🚨 문제 사항: 카테고리 필터 기능을 구현하는 과정에서 백엔드에서는 카테고리를 사용자가 직접 관리하는 구조였지만, 프론트엔드에서는 서비스 이용자가 카테고리를 선택만 하도록 제한된 UX를 제공하고자 함. 또한 카테고리 변경 시마다 중복 API 요청이 발생해 로딩 시간이 길어지는 문제가 발생함.

💡 해결 방안: 서비스 이용자 화면과 분리된 임시 관리자 페이지를 구현해 카테고리 편집은 관리자만 가능하도록 구조를 분리함. 또한 카테고리 데이터를 각 컴포넌트에서 개별적으로 fetch하던 구조를 개선하기 위해 전역 상태로 관리하고 fetch 로직을 상위 컴포넌트로 통합하여 중복 API 요청을 제거함.

😄 배운 점: 백엔드 구조를 그대로 따르기보다, 사용자 경험을 고려해 프론트엔드에서 구조를 재정리하는 판단이 중요하다는 점. 또한 상태 관리에 따라 성능과 사용성이 크게 달라질 수 있음을 체감함.

2. Mando

투두 리스트 활용을 통해 만다라트 계획표 실천을 도와주는 서비스

개발 기간

2024.01.30 ~ 2024.03.20

팀원 구성

기획자 1, 디자이너 2, FE 2, BE 2

사용 스택

- HTML, CSS module, SCSS, JavaScript, React, Axios
- VScode, Git, Github, Figma, Notion

배포 URL

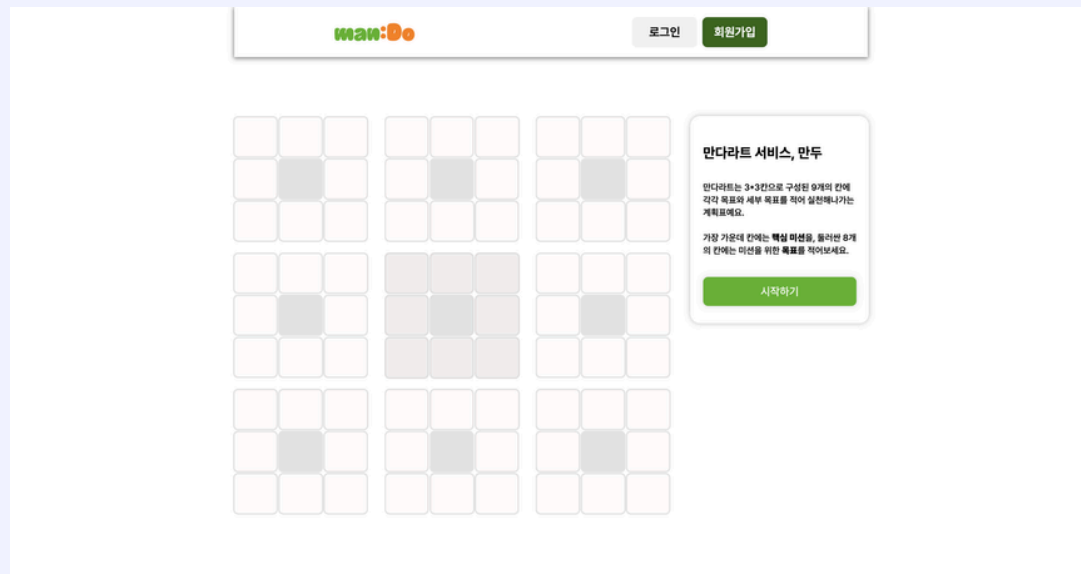
<https://mandomando.swygbro.com/mando>

Github

<https://github.com/swyp-3th-8team/frontend>



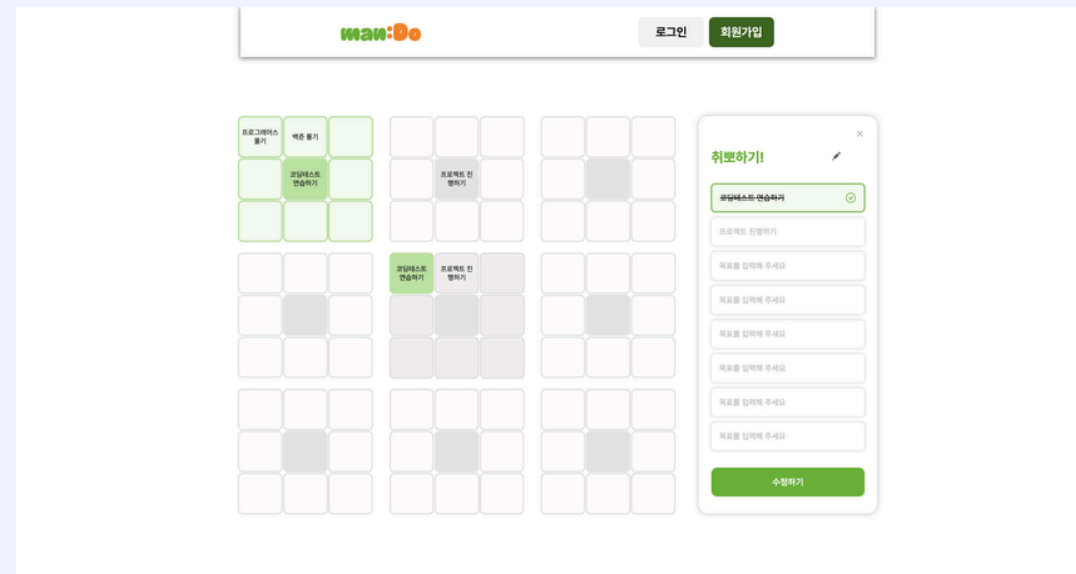
2. Mando



미리보기 페이지

: 로그인 이전 만두 서비스를 미리 알아보고
체험해 볼 수 있는 페이지

- 만다라트 창 클릭 → 로그인 페이지로 이동할 수 있는 모달창이 생성되고, 이후 로그인 페이지로 이동
- 시작하기 버튼 클릭 → 회원가입 페이지로 바로 이동



메인 페이지

: 만다라트의 가장 큰 목표 및 주제와 메인 목표를 이루기 위한 8가지 하위목표를 입력할 수 있는 페이지

- 메인 만다라트 창 클릭 → 우측에 주제, 하위 목표를 입력하고 수정할 수 있는 패널 생성



상세 페이지 (모달창)

: 각 하위목표를 달성하기 위한 계획과 다짐을 입력할 수 있는 모달창

- 각 메인 목표 클릭 시 창 독립 생성
- 상세 목표 칸 클릭 시 계획 및 다짐을 입력하고 수정할 수 있는 패널 생성
- 체크박스로 목표 달성 시 표기 가능

2. Mando

작업 기여도

1. SCSS를 활용한 스타일링 최적화

: CSS 전처리기인 SCSS를 도입하여 스타일 시트의 유지보수성과 가독성을 향상시킴. SCSS의 변수, 중첩, 모듈화 기능을 적극 활용해 코드의 재사용성을 높이고, 프로젝트 규모가 커짐에 따라 발생할 수 있는 스타일링 복잡성을 효과적으로 관리함.

2. useState를 통한 상태 관리

: 컴포넌트 내부에서 동적인 상태 변화를 관리하기 위해 useState hooks를 적극 활용함. 이를 통해 사용자 입력, API 호출 결과 등 다양한 상태를 효율적으로 관리함.

트러블 슈팅

1. API 연동 시 데이터 불일치 및 오류 처리 문제

🚨 문제 사항: 백엔드 API와의 연동 과정에서 데이터 형식 불일치 및 오류 응답에 대한 처리가 미흡하여 클라이언트 측에서 예기치 않은 동작이 발생함.

💡 해결 방안: API 명세를 시각화하여 정리해 상세히 작성하고, 이를 기반으로 클라이언트와 서버 간의 데이터 형식을 일치시킴.

2. 상태 관리로 인한 불필요한 렌더링 문제

🚨 문제 사항: 상태 관리 시 불필요한 상태 변화를 트리거하여 컴포넌트가 과도하게 렌더링되는 문제가 발생

💡 해결 방안: 상태 관리 시 실제로 UI에 영향을 주는 상태만을 관리하도록 코드를 개선하였으며, 필요에 따라 useMemo와 useCallback hooks를 활용하여 함수나 값을 메모이제이션하여 불필요한 렌더링을 방지함.

1. Omok

규칙을 직접 해석하고, 승리 조건을 알고리즘으로 구현한 오목 게임 서비스

개발 기간
(진행 중)

개요

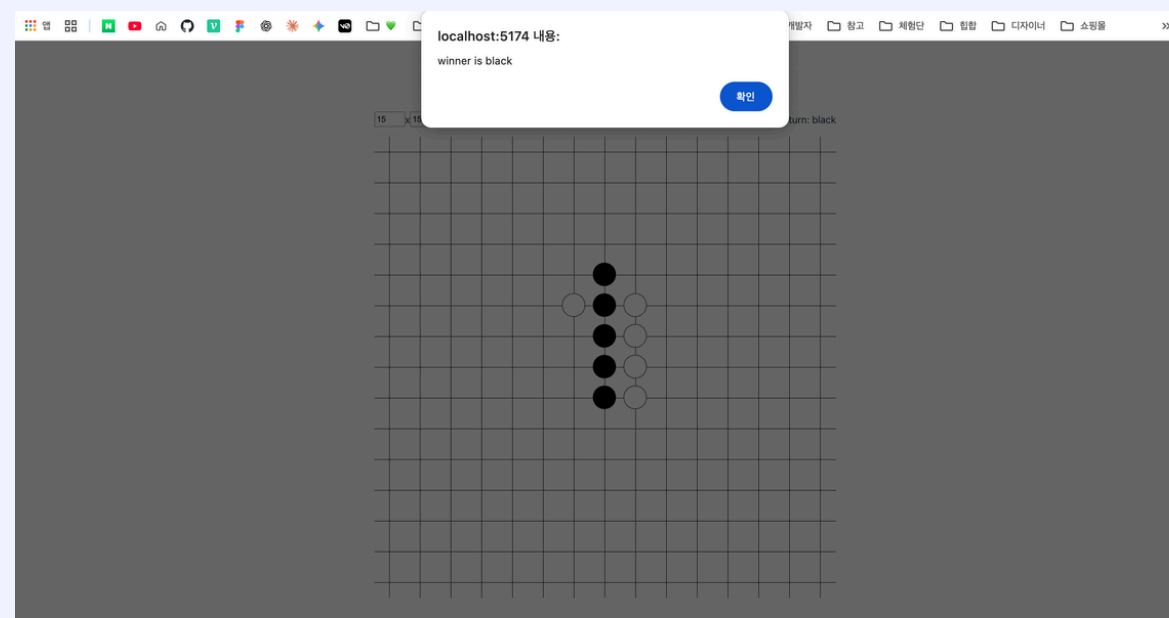
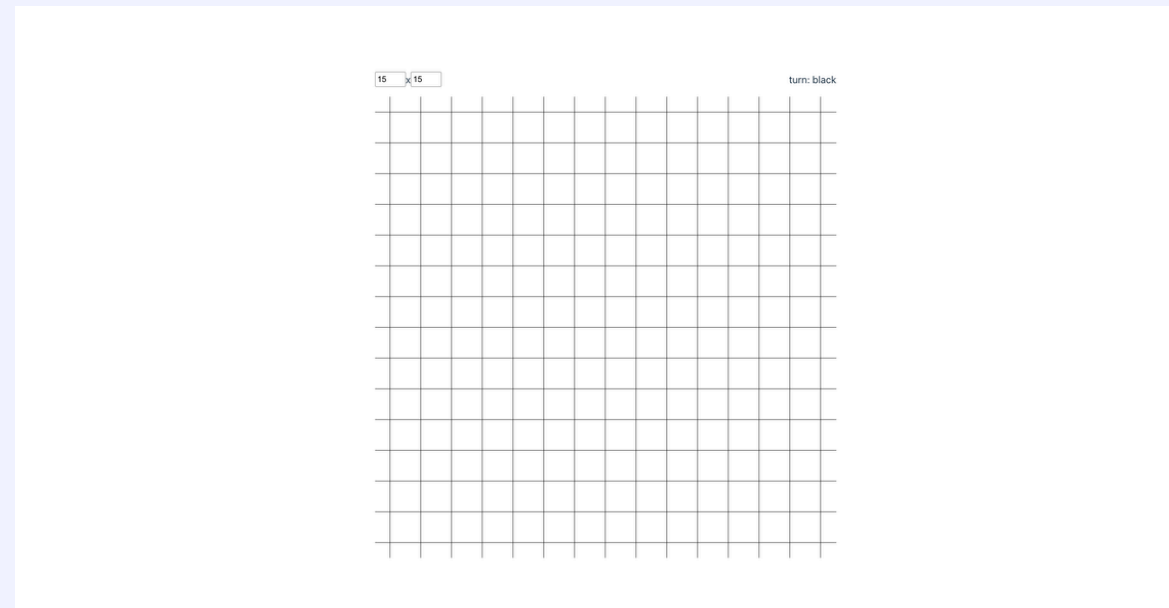
생성형 AI의 도움 없이 렌주를 기반으로 제작한
오목 게임 서비스

사용 스택

- HTML, CSS module, TypeScript, React
- VS code, Git, Github

Github

<https://github.com/jisangjiha/omok>



메인 페이지

- 사용자가 원하는 사이즈로
오목판 조정 가능
- 현재 차례 표시

기능

- 승리 조건에 맞춰 alert
- 이미 돌이 놓여진 경우 alert

1. Omok

기획 의도

게임 운영 프로젝트를 통해 문제를 정의하고 로직으로 풀어내는 과정에 집중하고자 했습니다. 특히 생성형 AI에 의존하지 않고 스스로 규칙을 분석하고 해결할 수 있는지 점검해보고자 구현 과정에서는 AI 도구를 사용하지 않았습니다. 막히는 부분에서는 구글 검색과 개발자 지인의 피드백을 통해 접근 방향을 점검하며 문제 해결 능력을 강화했습니다.

트러블 슈팅

오목 승리 룰 구현

🚩 문제 사항: 개발 초반, 오목의 승리 조건을 단순히 “같은 색 돌 5개”로 생각하여 제대로 된 룰이 구현되지 않음.

💡 해결 방법: 승리 조건을 “현재 돌 기준 연속된 방향으로 같은 색 돌 5개”로 재정의하였고, 가로·세로·대각선 총 4개의 방향 벡터를 기준으로 각 방향에 대해 양쪽으로 확장 탐색하며 연속된 돌의 개수를 계산하는 방식으로 구현함.

😁 배운 점: 프론트엔드 개발에서 UI 구현뿐 아니라 알고리즘적 사고와 상태 설계가 충분히 중요하다는 점을 체감.

확장 방향

오목 보드 UI 사용성 개선

- 현재 돌 위치 및 마지막 수를 명확히 표시하여 게임 진행 흐름을 직관적으로 인지할 수 있도록 개선

렌주룰(3-3, 4-4) 적용을 통한 규칙 고도화

- 단순 승리 판별 외, 금수 조건(3-3, 4-4)을 사전에 판별하는 로직을 추가하여 실제 오목 규칙에 가까운 구현을 목표로 함
- 기존 방향 탐색 로직을 확장하여 연속 여부뿐 아니라 열린 수, 닫힌 수를 구분하는 알고리즘 설계를 시도할 예정

2. do it;

Todo 리스트의 완료 상태를 한 눈에 확인할 수 있는 서비스

개발 기간

2025.04.03 ~ 2025.04.09

사용 스택

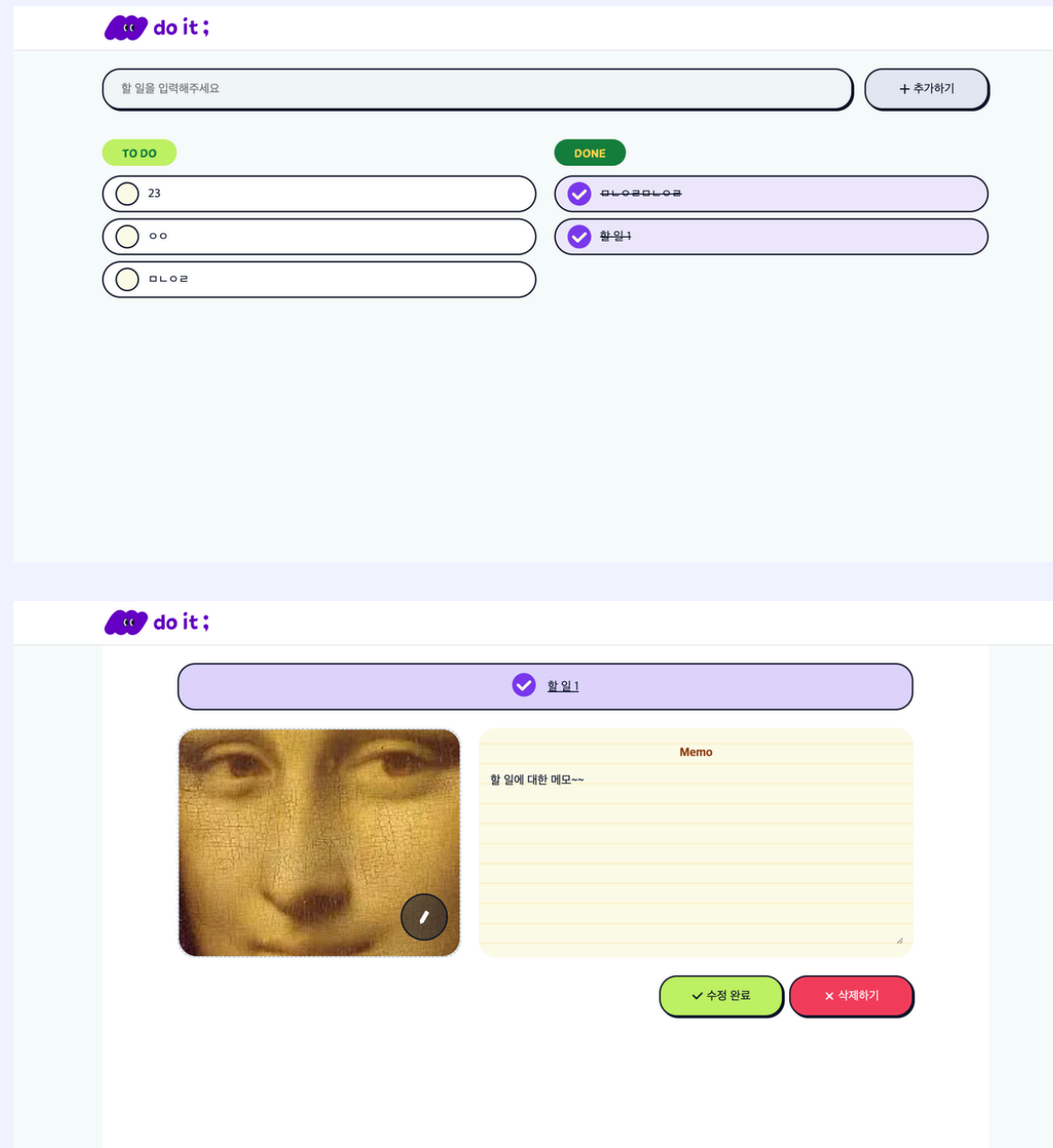
- HTML, CSS module, TypeScript, Next.js
- Vercel, VS code, Git, Github

Github

<https://github.com/jisangjiha/codeit-prework>

배포 링크

<https://codeit-prework.vercel.app/>



메인 페이지

- 할 일 추가하기
- TO DO / DONE 상태 표시

상세 페이지

- 할 일 상태, 타이틀 수정
- 이미지 및 메모 추가

2. do it;

작업 기여도

1. UI 동작 로직 및 사용자 경험 개선

: 기능 단위로 상태 변화를 시각적으로 표현하고, 예외 상황을 처리하여 안정적인 사용자 경험을 구현함.

2. 반응형 레이아웃 구현

: CSS Flexbox와 미디어쿼리를 활용해 다양한 해상도에서도 일관된 인터페이스를 유지하도록 함.

트러블 슈팅

입력 데이터 초기화 문제

🚩 문제 사항: 폼 제출 시, 입력 데이터가 초기화되지 않는 문제가 발생함.

💡 해결 방법: 폼 제출 후 각 입력 필드의 상태를 초기 상태로 재설정하도록 useState 혹은 상태 업데이트 함수를 호출하여 문제를 해결함. 이를 통해 폼 제출 후 입력 필드가 정상적으로 초기화되도록 함.

🧐 배운 점: 스토리지에 대한 이해도

3. 베이글 프로젝트

미국의 Ess-a-Bagel을 참고하여 만든 커스텀 베이글을 주문할 수 있는 서비스

개발 기간

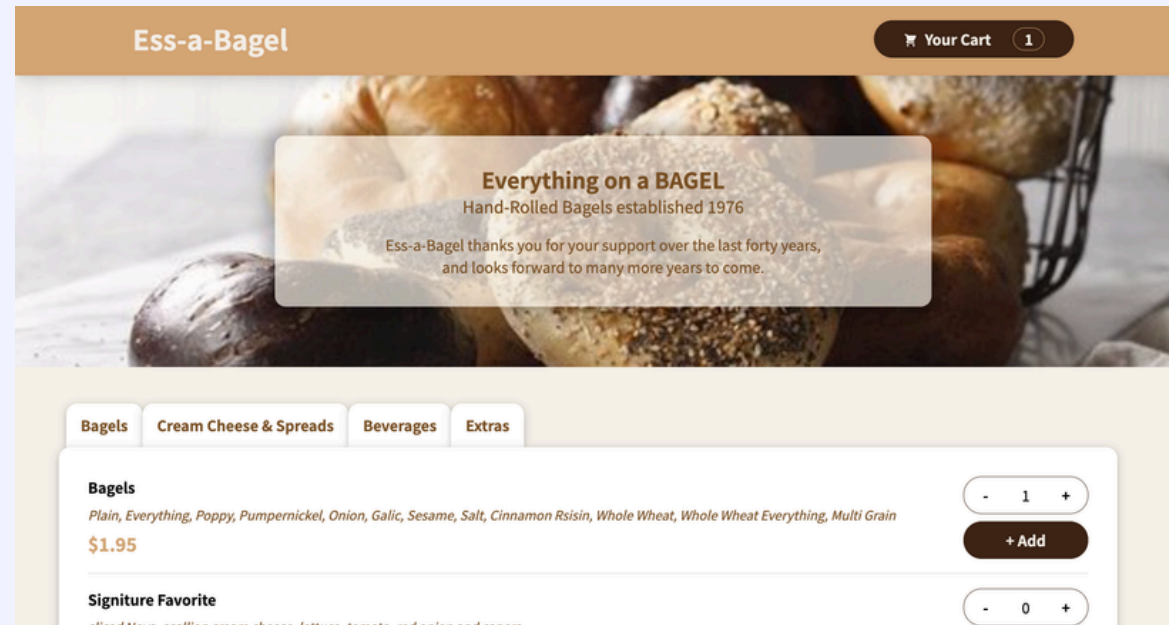
2023.09.20 ~ 2023.12.28

사용 스택

- HTML, CSS module, JavaScript, React
- VScode, Git, Github

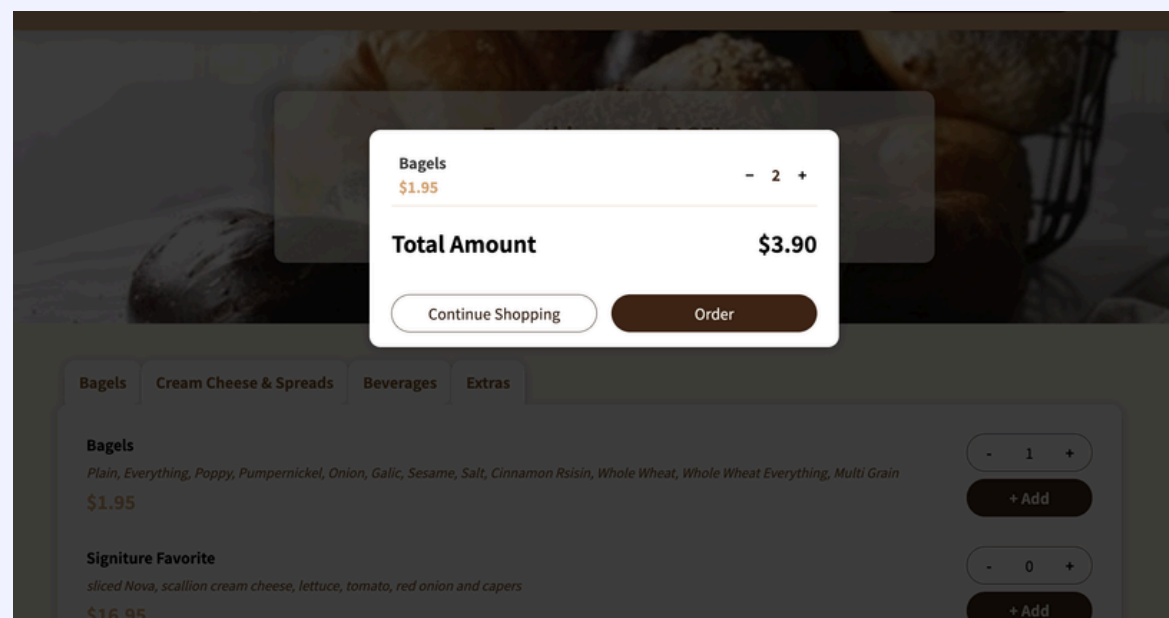
Github

<https://github.com/jisangjiha/bagel-project>



메인 페이지

- 홈페이지 소개
- 각 메뉴 탭 별로 상세 메뉴의 설명과 가격 확인 가능



모달창(장바구니)

- 총 주문 금액 확인 가능
- 담은 메뉴 수량 변경 가능

3. 베이글 프로젝트

작업 기여도

1. 컴포넌트 기반 아키텍처 설계

: React의 컴포넌트 기반 아키텍처를 활용하여 UI 요소들을 독립적인 컴포넌트로 분리해 코드의 재사용성을 높임

2. React Context API를 통한 전역 상태 관리

: React의 useContext 훅을 활용하여 전역 상태를 관리함. 이를 통해 컴포넌트 간의 상태 공유를 용이하게 하고, 복잡한 props drilling을 피하고자 함

트러블 슈팅

1. 장바구니 항목의 수량 동기화 문제

🚨 문제 사항: 장바구니 항목의 수량을 변경할 때, 상태 업데이트 지연으로 UI와 실제 데이터 간의 불일치가 발생하는 문제가 발생함

💡 해결 방안: useEffect 훅을 활용해 수량 변경 시마다 상태를 즉시 업데이트하고, 이를 UI에 반영하도록 함

2. 컴포넌트 리렌더링으로 인한 성능 저하

🚨 문제 사항: 상태 변경으로 인해 불필요한 컴포넌트 리렌더링이 발생하여 성능 저하가 우려됨

💡 해결 방안: React.memo를 활용해 컴포넌트를 메모이제이션하고, useCallback과 useMemo 훅을 사용해 함수와 값을 캐싱함으로써 불필요한 리렌더링을 방지함

3. 베이글 프로젝트

리팩토링

Tailwind CSS 도입

CSS Module로 스타일 관리를 하였는데 UI 확장 시 일관된 스타일 적용과 코드 가독성과 효율성을 개선하고 싶어 도입하게 됨

⚙️ 과정

- 기존 컴포넌트들의 스타일을 Tailwind 유틸리티 클래스로 점진적 전환
- 반복되거나 복잡한 스타일은 Tailwind @apply를 활용해 리팩토링
- 컬러나 폰트 등 디자인 시스템은 tailwind.config.js에 커스터마이징

🔑 결과

- 스타일 코드 간소화 → CSS 코드량 약 30% 감소
- 재사용성 증가: 공통 Tailwind 클래스 묶은 @apply 스타일, 후속 기능 추가 시 활용
- 생산성 향상: CSS 재컴파일 없이 빠르게 UI 수정 가능

🔍 아쉬웠던 점

- JSX의 className이 길어져 가독성 저하 우려

+ 이외 추가 기능 개선

- 주문 완료 Toast 추가: 명확한 시각적 피드백을 제공함으로써 불필요한 재주문을 방지하고 사용자 경험 개선
- 주문 내역 확인 기능 추가: 장바구니에서 주문 후 내역 확인할 수 있는 버튼/모달 생성

감사합니다